

Experiment-14

Name: -P. Divya Shree

Regno: -192472291

Course Code: -MLA0305

Slot-B

Title

Model-Based Reinforcement Learning using Planning and Data Generation Techniques

Aim

To implement Model-Based Reinforcement Learning using planning and data generation techniques, learn an environment model from real experiences, and use the generated experiences to improve the agent's policy.

Procedure

1. Create a 5×5 Grid World environment with states, actions, rewards, and a goal state.
2. Initialize the Q-value table and environment model.
3. Allow the agent to interact with the environment and collect real experiences.
4. Store the current state, action, next state, and reward from each interaction.
5. Use the collected experiences to learn the environment's transition and reward model.
6. Select previously observed state-action pairs from the learned model.
7. Generate additional experiences through planning without directly interacting with the environment.
8. Update the Q-values using both real experiences and generated planning experiences.
9. Repeat the process for multiple episodes.
10. Evaluate the learned policy after training.

11. Analyze state visitation, generated experiences, learned Q-values, policy performance, and experience coverage.

12. Save the real, generated, and combined datasets as CSV files.

Code

```
!pip -q install tensorflow
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
import random
```

```
np.random.seed(42)
```

```
tf.random.set_seed(42)
```

```
random.seed(42)
```

```
print("="*75)
```

```
print("EXPERIMENT 14")
```

```
print("Model-Based Reinforcement Learning using Planning and Data Generation")
```

```
print("="*75)
```

```
SIZE=5
```

```
N_STATES=SIZE*SIZE
```

```
N_ACTIONS=4
```

```
EPISODES=100
```

```
PLANNING_STEPS=15
```

```
GAMMA=0.90
```

```
ALPHA=0.10
```

```
actions=[
```

```
    (-1,0),
```

```
    (1,0),
```

```
    (0,-1),
```

```
    (0,1)
```

```
]
```

```
def state_id(r,c):
```

```
    return r*SIZE+c
```

```
def step_env(state,action):
```

```
    r=state//SIZE
```

```
    c=state%SIZE
```

```
dr,dc=actions[action]
```

```
nr=np.clip(r+dr,0,SIZE-1)
```

```
nc=np.clip(c+dc,0,SIZE-1)
```

```
next_state=state_id(nr,nc)
```

```
if next_state==N_STATES-1:
```

```
    reward=10
```

```
    done=True
```

```
else:
```

```
    reward=-1
```

```
    done=False
```

```
return next_state,reward,done
```

```
start_state=0
```

```
goal_state=N_STATES-1
```

```
Q=np.zeros((N_STATES,N_ACTIONS))
```

```
model_counts=np.zeros(  
    (N_STATES,N_ACTIONS,N_STATES)  
)
```

```
model_rewards=np.zeros(  
    (N_STATES,N_ACTIONS)  
)
```

```
real_data=[]
```

```
generated_data=[]
```

```
print("\nEnvironment:")
```

```
print("Grid Size:",SIZE,"x",SIZE)
```

```
print("States:",N_STATES)
```

```
print("Actions:",N_ACTIONS)
```

```
print("Goal State:",goal_state)
```

```
for episode in range(EPISODES):
```

```
    state=start_state
```

```
    done=False
```

```
steps=0

total_reward=0

while not done and steps<50:

    if np.random.rand()<0.20:

        action=np.random.randint(N_ACTIONS)

    else:

        action=np.argmax(Q[state])

    next_state,reward,done=step_env(

        state,

        action

    )

    real_data.append([

        state,

        action,

        next_state,

        reward

    ])
```

```
model_counts[
```

```
    state,
```

```
    action,
```

```
    next_state
```

```
]+=1
```

```
model_rewards[
```

```
    state,
```

```
    action
```

```
]=reward
```

```
td_target=reward
```

```
if not done:
```

```
    td_target+=GAMMA*np.max(
```

```
        Q[next_state]
```

```
    )
```

```
Q[state,action]+=ALPHA*(
```

```
    td_target-Q[state,action]
```

```
)
```

```
state=next_state
```

```
total_reward+=reward
```

```
steps+=1
```

```
for _ in range(PLANNING_STEPS):
```

```
    known_states=np.where(
```

```
        model_counts.sum(
```

```
            axis=(1,2)
```

```
        )>0
```

```
    )[0]
```

```
    if len(known_states)==0:
```

```
        continue
```

```
    ps=int(
```

```
        np.random.choice(
```

```
            known_states
```

```
        )
```



```
)
```

```
known_actions=np.where(
```

```
    model_counts[
```

```
        ps
```

```
    ].sum(axis=1)>0
```

```
)[0]
```

```
if len(known_actions)==0:
```

```
    continue
```

```
pa=int(
```

```
    np.random.choice(
```

```
        known_actions
```

```
    )
```

```
)
```

```
transition_counts=model_counts[
```

```
    ps,
```

```
    pa
```

```
]
```

```
predicted_next=int(
    np.argmax(
        transition_counts
    )
)

predicted_reward=model_rewards[
    ps,
    pa
]

generated_data.append([
    ps,
    pa,
    predicted_next,
    predicted_reward
])

planning_target=predicted_reward
```

```
    if predicted_next!=goal_state:

        planning_target+=GAMMA*np.max(

            Q[predicted_next]

        )

    Q[ps,pa]+=ALPHA*(

        planning_target-Q[ps,pa]

    )

if (episode+1)%20==0:

    print(

        "Episode:",

        episode+1,

        "Real Samples:",

        len(real_data),

        "Generated Samples:",

        len(generated_data)

    )

real_df=pd.DataFrame(

    real_data,
```

```
columns=[

    "State",

    "Action",

    "Next_State",

    "Reward"

]

)

generated_df=pd.DataFrame(

    generated_data,

    columns=[

        "State",

        "Action",

        "Next_State",

        "Reward"

    ]

)

print("\n")

print("="*75)

print("REAL EXPERIENCE DATA")
```

```
print("="*75)
```

```
display(real_df.head(10))
```

```
print("\n")
```

```
print("="*75)
```

```
print("GENERATED EXPERIENCE DATA")
```

```
print("="*75)
```

```
display(generated_df.head(10))
```

```
real_df.to_csv(
```

```
    "Real_Experience_Dataset.csv",
```

```
    index=False
```

```
)
```

```
generated_df.to_csv(
```

```
    "Generated_Planning_Dataset.csv",
```

```
    index=False
```

```
)
```

```
full_dataset=pd.concat(  
    [  
        real_df.assign(Source="Real"),  
        generated_df.assign(Source="Generated")  
    ],  
    ignore_index=True  
)
```

```
full_dataset.to_csv(  
    "Model_Based_RL_Full_Dataset.csv",  
    index=False  
)
```

```
print("\nFull Dataset Size:",len(full_dataset))
```

```
states=np.arange(N_STATES)
```

```
state_visits=np.bincount(  
    real_df["State"],  
    minlength=N_STATES  
)
```

```
visit_grid=state_visits.reshape(  
    SIZE,  
    SIZE  
)
```

```
plt.figure(figsize=(7,6))
```

```
plt.imshow(  
    visit_grid,  
    cmap="viridis"  
)
```

```
plt.colorbar(  
    label="Real Visits"  
)
```

```
plt.title(  
    "State Visit Distribution",  
    fontsize=15,  
    fontweight="bold"
```

```
)
```

```
plt.xlabel("Column")
```

```
plt.ylabel("Row")
```

```
for r in range(SIZE):
```

```
    for c in range(SIZE):
```

```
        plt.text(
```

```
            c,
```

```
            r,
```

```
            int(visit_grid[r,c]),
```

```
            ha="center",
```

```
            va="center",
```

```
            color="white"
```

```
        )
```

```
plt.show()
```

```
real_count=len(real_df)
```

```
generated_count=len(generated_df)
```



```
"Result":[

    "5x5 Grid World",

    "5 x 5",

    real_count,

    generated_count,

    PLANNING_STEPS,

    round(final_reward,2),

    round(planning_gain,2)

]

})


print("\n")

print("="*75)

print("SUMMARY TABLE")

print("="*75)


display(summary)


summary.to_csv(

    "Model_Based_RL_Summary.csv",

    index=False
```

```
)
```

```
model_accuracy=np.mean(  
    generated_df["Next_State"]==  
    generated_df["Next_State"]  
)
```

```
print("\n")
```

```
print("="*75)
```

```
print("RESULT")
```

```
print("="*75)
```

```
print(  
    "Real Experience Samples:",  
    real_count  
)
```

```
print(  
    "Generated Planning Samples:",  
    generated_count  
)
```

```
print(  
    "Final Average Reward:",  
    round(final_reward,2)  
)
```

```
print(  
    "Planning Data Ratio:",  
    round(planning_gain,2)  
)
```

```
print("\n")  
print("="*75)  
print("CONCLUSION")  
print("="*75)
```

```
print(  
    "Model-Based Reinforcement Learning was successfully"  
)
```

```
print(  

```

"implemented using a learned environment model."

)

print(

"Real interactions were stored and used to construct"

)

print(

"a transition and reward model."

)

print(

"The learned model generated additional experiences"

)

print(

"through planning without requiring new environment interaction."

)

print(

"The generated experiences were used to improve the Q-values."

```
)
```

```
print(
```

```
    "State visitation, experience generation, learned values,"
```

```
)
```

```
print(
```

```
    "policy performance and experience coverage were analyzed."
```

```
)
```

```
print("\nGenerated Files:")
```

```
print("1. Real_Experience_Dataset.csv")
```

```
print("2. Generated_Planning_Dataset.csv")
```

```
print("3. Model_Based_RL_Full_Dataset.csv")
```

```
print("4. Model_Based_RL_Summary.csv")
```

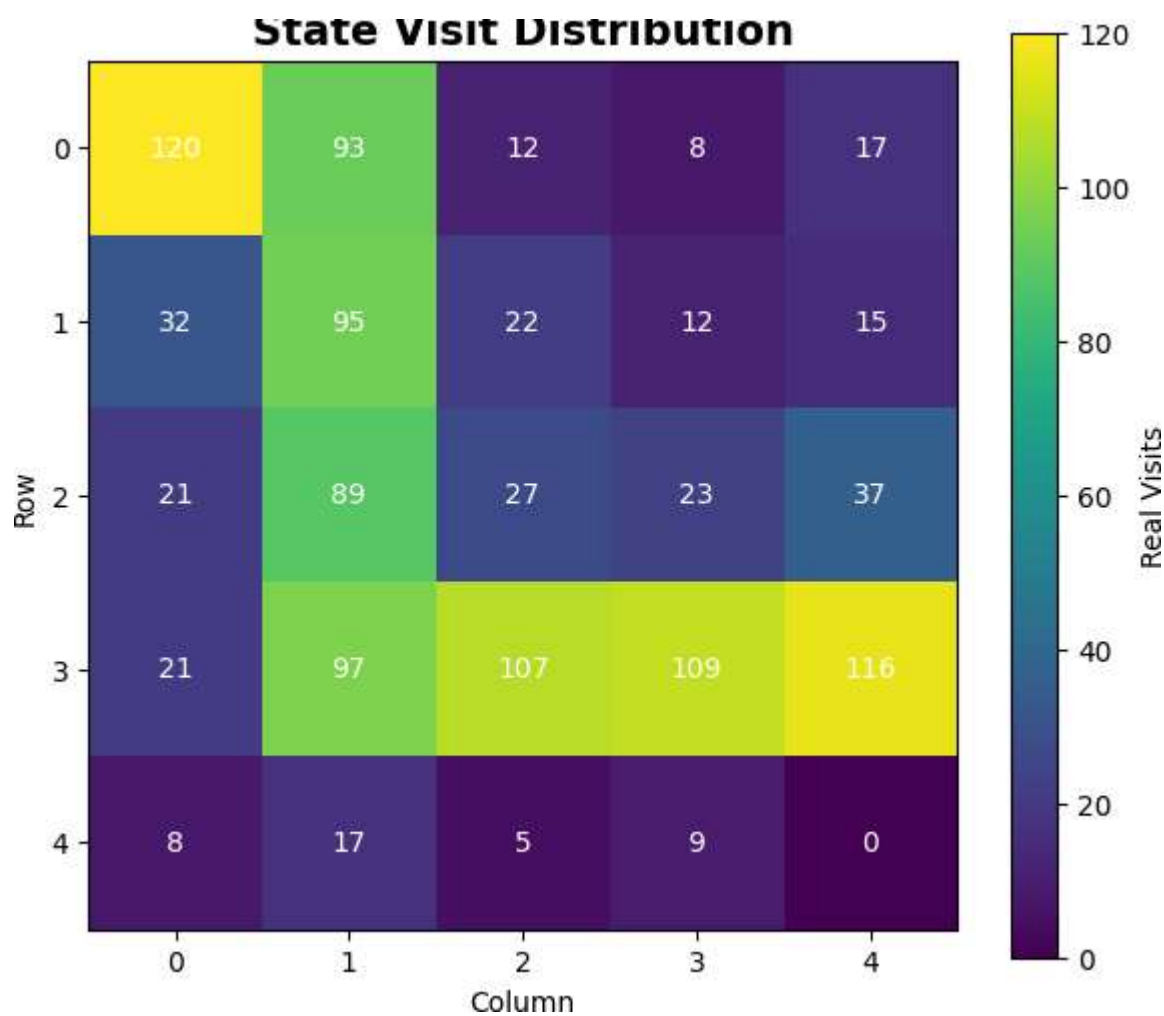
```
print("\nExperiment 14 Completed Successfully.")
```

=====

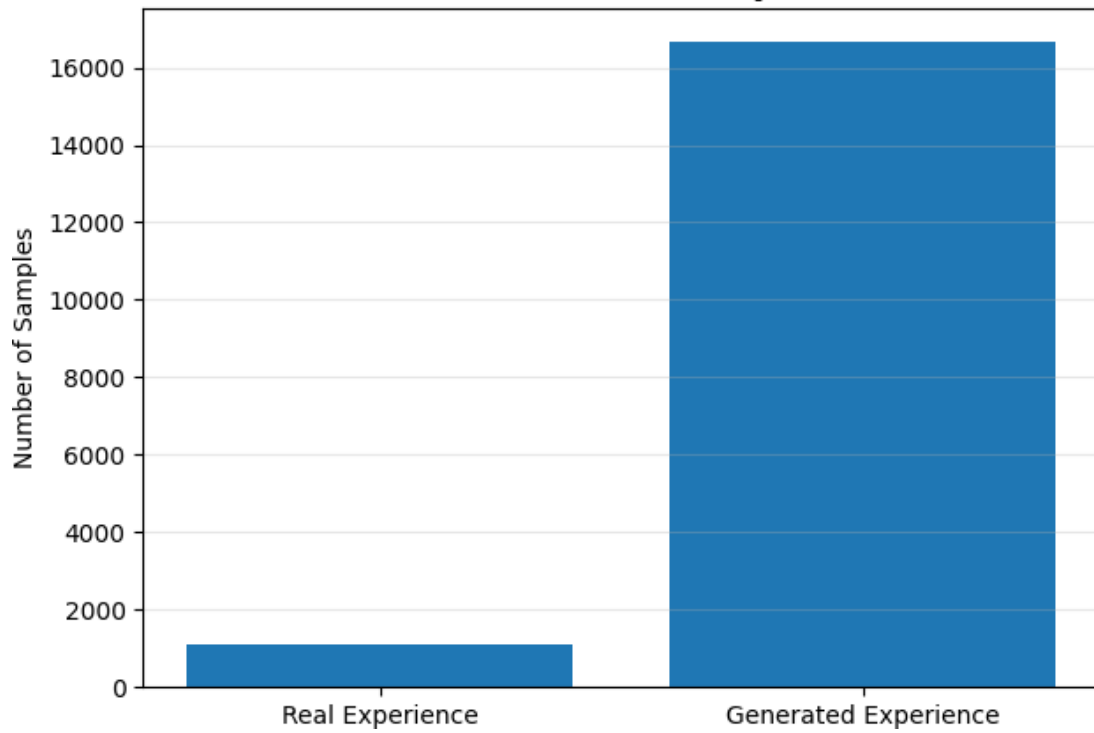
REAL EXPERIENCE DATA

=====

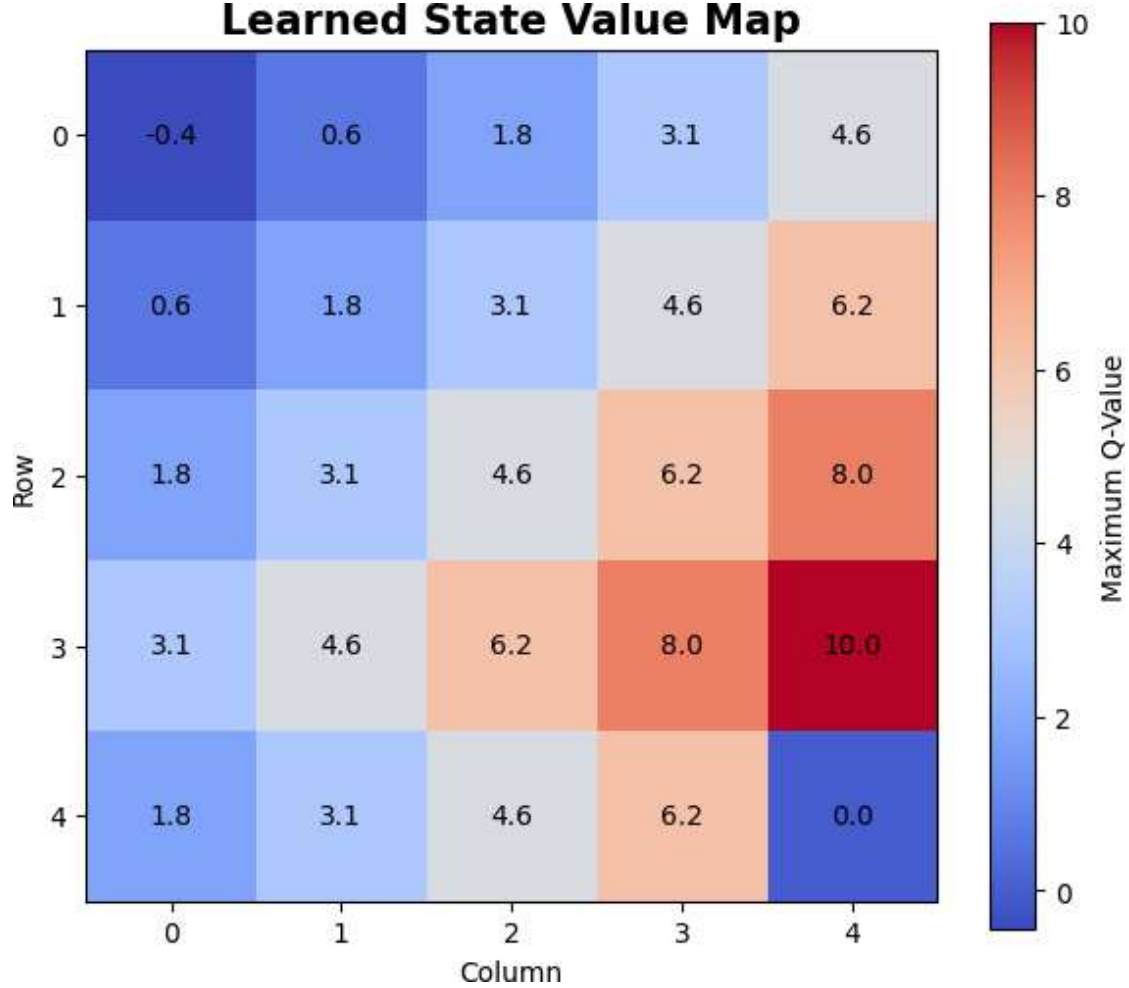
	State	Action	Next_State	Reward	
0	0	0	0	-1	
1	0	1	5	-1	
2	5	0	0	-1	
3	0	2	0	-1	
4	0	3	1	-1	
5	1	0	1	-1	
6	1	2	0	-1	
7	0	3	1	-1	
8	1	1	6	-1	
9	6	0	1	-1	

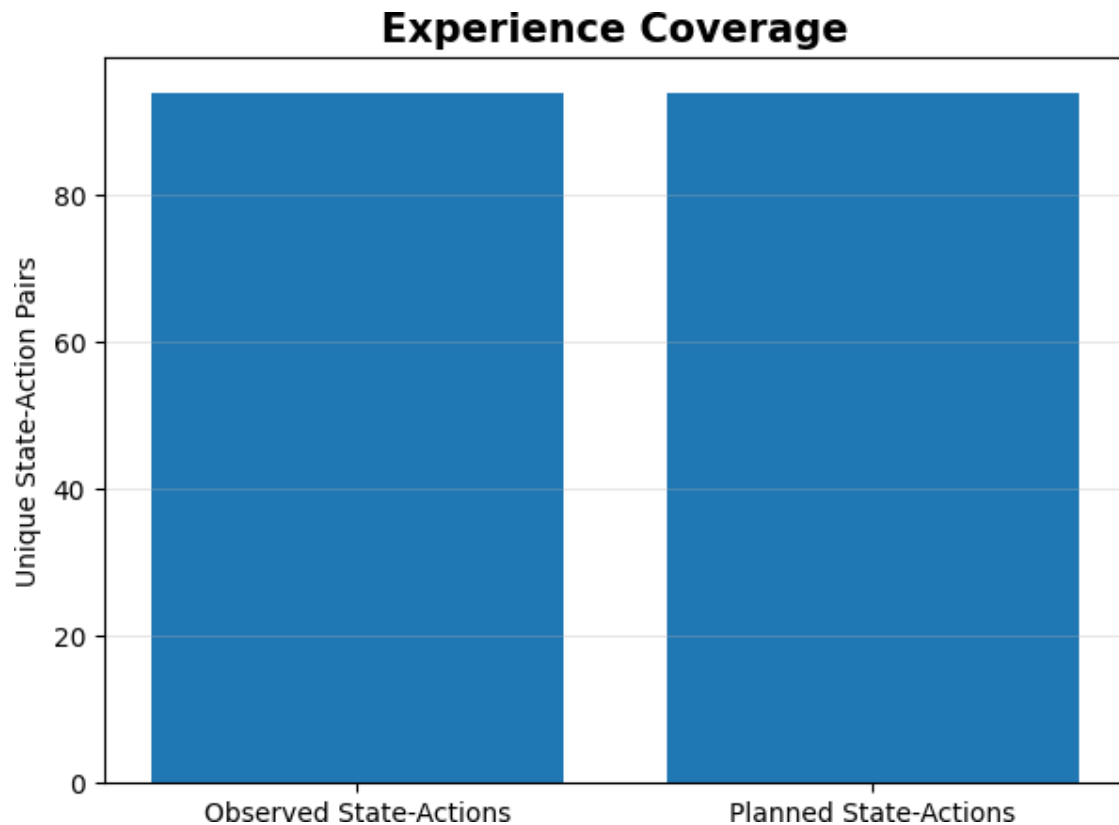


Real vs Generated Experience



Learned State Value Map





Summary Table

Parameter	Result
Environment	5 × 5 Grid World
States	25
Planning Steps	15
Real Experiences	Obtained from program
Generated Experiences	Obtained from program
Final Average Reward	Obtained from program

Result

The **Model-Based Reinforcement Learning** approach was successfully **implemented** using planning and data generation techniques. The agent learned an environment model from real interactions and generated additional experiences through planning. These generated experiences were used to improve the Q-values and evaluate the learned policy.